Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3580305.3599347

RESEARCH-ARTICLE

Federated Few-shot Learning

SONG WANG, University of Virginia, Charlottesville, VA, United States

XINGBO FU, University of Virginia, Charlottesville, VA, United States

KAIZE DING, Arizona State University, Tempe, AZ, United States

CHEN CHEN, University of Virginia, Charlottesville, VA, United States

HUIYUAN CHEN, Case Western Reserve University, Cleveland, OH, United States

JUNDONG LI, University of Virginia, Charlottesville, VA, United States

Open Access Support provided by:

Arizona State University

University of Virginia

Case Western Reserve University

PDF Download
3580305.3599347.pdf
22 January 2026
Total Citations: 31
Total Downloads: 1798

Published: 06 August 2023

Citation in BibTeX format

KDD '23: The 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining
August 6 - 10, 2023
CA, Long Beach, USA

Conference Sponsors:
SIGMOD
SIGKDD

KDD '23: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (August 2023)
https://doi.org/10.1145/3580305.3599347
ISBN: 9798400701030

Federated Few-shot Learning

Song Wang
University of Virginia
sw3wv@virginia.edu

Xingbo Fu
University of Virginia
xf3av@virginia.edu

Kaize Ding
Arizona State University
kaize.ding@asu.edu

Chen Chen
University of Virginia
zrh6du@virginia.edu

Huiyuan Chen
Case Western Reserve University
hxc501@case.edu

Jundong Li
University of Virginia
jundong@virginia.edu

ABSTRACT

Federated Learning (FL) enables multiple clients to collaboratively learn a machine learning model without exchanging their own local data. In this way, the server can exploit the computational power of all clients and train the model on a larger set of data samples among all clients. Although such a mechanism is proven to be effective in various fields, existing works generally assume that each client preserves sufficient data for training. In practice, however, certain clients may only contain a limited number of samples (i.e., few-shot samples). For example, the available photo data taken by a specific user with a new mobile device is relatively rare. In this scenario, existing FL efforts typically encounter a significant performance drop on these clients. Therefore, it is urgent to develop a few-shot model that can generalize to clients with limited data under the FL scenario. In this paper, we refer to this novel problem as *federated few-shot learning*. Nevertheless, the problem remains challenging due to two major reasons: the global data variance among clients (i.e., the difference in data distributions among clients) and the local data insufficiency in each client (i.e., the lack of adequate local data for training). To overcome these two challenges, we propose a novel federated few-shot learning framework with two separately updated models and dedicated training strategies to reduce the adverse impact of global data variance and local data insufficiency. Extensive experiments on four prevalent datasets that cover news articles and images validate the effectiveness of our framework compared with the state-of-the-art baselines. Our code is provided[1].

CCS CONCEPTS

• **Computing methodologies → Distributed algorithms**.

KEYWORDS

Federated Learning; Few-shot Learning; Knowledge Distillation

ACM Reference Format:
Song Wang, Xingbo Fu, Kaize Ding, Chen Chen, Huiyuan Chen, and Jundong Li. 2023. Federated Few-shot Learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23), August 6–10, 2023, Long Beach, CA, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3580305.3599347

[1]https://github.com/SongW-SW/F2L

This work is licensed under a Creative Commons Attribution International 4.0 License.

KDD '23, August 6–10, 2023, Long Beach, CA, USA
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0103-0/23/08.
https://doi.org/10.1145/3580305.3599347

1 INTRODUCTION

The volume of valuable data is growing massively with the rapid development of mobile devices [4, 34]. Recently, researchers have developed various machine learning methods [5, 57, 61] to analyze and extract useful information from such large-scale real-world data. Among these methods, Federated Learning (FL) is an effective solution, which aims to collaboratively optimize a centralized model over data distributed across a large number of clients [7, 13, 22, 62]. In particular, FL trains a global model on a server by aggregating the local models learned on each client [2]. Moreover, by avoiding the direct exchange of private data, FL can provide effective protection of local data privacy for clients [31]. As an example, in Google Photo Categorization [12, 33], the server aims to learn an image classification model from photos distributed among a large number of clients, i.e., mobile devices. In this case, FL can effectively conduct learning tasks without revealing private photos to the server.

In fact, new learning tasks (e.g., novel photo classes) are constantly emerging over time [50, 59]. In consequence, FL can easily encounter a situation where the server needs to solve a new task with limited available data as the reference. In the previous example of Google Photo Categorization, as illustrated in Fig. 1, the server may inevitably need to deal with novel photo classes such as the latest electronic products, where only limited annotations are available. Nevertheless, existing FL works generally assume sufficient labeled samples for model training, which inevitably leads to unsatisfying classification performance for new tasks with limited labeled samples [14]. Therefore, to improve the practicality of FL in realistic scenarios, it is important to solve this problem by learning an FL model that can achieve satisfactory performance on new tasks with limited samples. In this paper, we refer to this novel problem setting as *federated few-shot learning*.

Recently, many few-shot learning frameworks [15, 44, 52, 55] have been proposed to deal with new tasks with limited samples. Typically, the main idea is to learn meta-knowledge from *base classes* with abundant samples (e.g., photo classes such as portraits). Then such meta-knowledge is generalized to *novel classes* with limited samples (e.g., photo classes such as new electronic products), where novel classes are typically disjoint from base classes. However, as illustrated in Fig. 1, it remains challenging to conduct few-shot learning under the federated setting due to the following reasons. First, due to the *global data variance* (i.e., the differences in data distributions across clients), the aggregation of local models on the server side will disrupt the learning of meta-knowledge in each

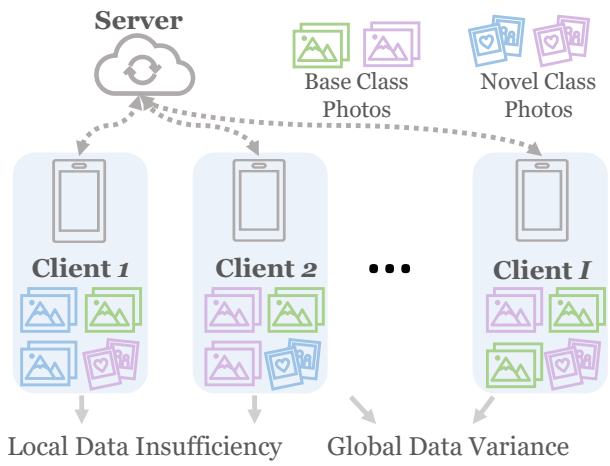


Figure 1: The two challenges of federated few-shot learning as an example in Google Photo Categorization: local data insufficiency and global data variance.

client [23]. Generally, the meta-knowledge is locally learned from different classes in each client and thus is distinct among clients, especially under the non-IID scenario, where the data variance can be even larger among clients compared with the IID scenario. Since the server will aggregate the local models from different clients and then send back the aggregated model, the learning of meta-knowledge in each client will be potentially disrupted. Second, due to the *local data insufficiency* in clients, it is non-trivial to learn meta-knowledge from each client. In FL, each client only preserves a relatively small portion of the total data [1, 13]. However, meta-knowledge is generally learned from data in a variety of classes [15, 52]. As a result, it is difficult to learn meta-knowledge from data with less variety, especially in the non-IID scenario, where each client only has a limited amount of classes.

To effectively solve the aforementioned challenges, we propose a novel **F**ederated **F**ew-shot **L**earning framework, named F^{2}L. First, we propose a decoupled meta-learning framework to mitigate the disruption from the aggregated model on the server. Specifically, the proposed framework retains a unique *client-model* for each client to learn meta-knowledge and a shared *server-model* to learn client-invariant knowledge (e.g., the representations of samples), as illustrated in Fig. 2. Specifically, the client-model in each client is updated locally and will not be shared across clients, while the server-model can be updated across clients and sent to the server for aggregation. Such a design decouples the learning of meta-knowledge (via client-model) from learning client-invariant knowledge (via server-model). In this way, we can mitigate the disruption from the aggregated model on the server caused by global data variance among clients. Second, to compensate for local data insufficiency in each client, we propose to leverage global knowledge learned from all clients with two dedicated update strategies. In particular, we first transfer the learned meta-knowledge in client-model to server-model by maximizing the mutual information between their output (i.e., *local-to-global knowledge transfer*). Then we propose a partial knowledge distillation strategy for each client to selectively extract useful knowledge from server-model (i.e., *global-to-local knowledge distillation*). In this manner, each client can leverage the beneficial knowledge in other clients to learn meta-knowledge from more data. In summary, our contributions are as follows:

- **Problem**. We investigate the challenges of learning meta-knowledge in the novel problem of federated few-shot learning from the perspectives of *global data variance* and *local data insufficiency*. We also discuss the necessity of tackling these challenges.
- **Method**. We develop a novel federated few-shot learning framework F^{2}L with three essential strategies: (1) a decoupled meta-learning framework to mitigate disruption from the aggregated model on the server; (2) mutual information maximization for local-to-global knowledge transfer; (3) a novel partial knowledge distillation strategy for global-to-local knowledge distillation.
- **Experiments**. We conduct experiments on four few-shot classification datasets covering both news articles and images under the federated scenario. The results further demonstrate the superiority of our proposed framework.

2 PRELIMINARIES

2.1 Problem Definition

In FL, given a set of I clients, i.e., $\{\mathbb{C}^{(i)}\}_{i=1}^{I}$, where I is the number of clients, each $\mathbb{C}^{(i)}$ owns a local dataset $\mathcal{D}^{(i)}$. The main objective of FL is to learn a global model over data across all clients (i.e., $\{\mathcal{D}^{(i)}\}_{i=1}^{I}$) without the direct exchange of data among clients. Following the conventional FL strategy [13, 32, 36], a server $\mathbb{S}$ will aggregate locally learned models from all clients for a global model.

Under the prevalent few-shot learning scenario, we consider a supervised setting in which the data samples for client $\mathbb{C}^{(i)}$ are from its local dataset: $(x, y) \in \mathcal{D}^{(i)}$, where x is a data sample, and y is the corresponding label. We first denote the entire set of classes on all clients as C. Depending on the number of labeled samples in each class, C can be divided into two categories: base classes C_b and novel classes C_n, where $C = C_b \cup C_n$ and $C_b \cap C_n = \emptyset$. In general, the number of labeled samples in C_b is sufficient, while it is generally small in C_n [15, 44]. Correspondingly, each local dataset can be divided into a base dataset $\mathcal{D}_b^{(i)} = \{(x, y) \in \mathcal{D}^{(i)} : y \in C_b\}$ and a novel dataset $\mathcal{D}_n^{(i)} = \{(x, y) \in \mathcal{D}^{(i)} : y \in C_n\}$. In the few-shot setting, the evaluation of the model generalizability to novel classes C_n is conducted on $\mathcal{D}_n^{(i)}$, which contains only limited labeled samples. The data samples in $\mathcal{D}_b^{(i)}$ will be used for training. Then we can formulate the studied problem of federated few-shot learning as follows:

Definition 1. ***Federated Few-shot Learning:*** *Given a set of I clients $\{\mathbb{C}^{(i)}\}_{i=1}^{I}$ and a server $\mathbb{S}$, federated few-shot learning aims to learn a global model after aggregating model parameters locally learned from $\mathcal{D}_b^{(i)}$ in each client such that the model can accurately predict labels for unlabeled samples (i.e., query set $\mathcal{Q}$) in $\mathcal{D}_n^{(i)}$ with only a limited number of labeled samples (i.e., support set $\mathcal{S}$).*

More specifically, if the support set $\mathcal{S}$ consists of exactly K labeled samples for each of N classes from $\mathcal{D}_n^{(i)}$, and the query set $\mathcal{Q}$ is sampled from the same N classes, the problem is defined as Federated N-way K-shot Learning. Essentially, the objective of federated

few-shot learning is to learn a globally shared model across clients that can be fast adapted to data samples in $\mathcal{D}_n^{(i)}$ with only limited labeled samples. Therefore, the crucial part is to effectively learn meta-knowledge from the base datasets $\{\mathcal{D}_b^{(i)}\}_{i=1}^{I}$ in all clients. Such meta-knowledge is generalizable to novel classes unseen during training and thus can be utilized to classify data samples in each $\mathcal{D}_n^{(i)}$, which consists of only limited labeled samples.

2.2 Episodic Learning

In practice, we adopt the prevalent episodic learning framework for model training and evaluation, which has proven to be effective in various few-shot learning scenarios [9, 10, 40, 52, 56]. Specifically, the model evaluation (i.e., meta-test) is conducted on a certain number of *meta-test tasks*, where each task contains a small number of labeled samples as references and unlabeled samples for classification. The local model training (i.e., meta-training) process in each client is similarly conducted on a specific number of *meta-training* tasks, where each task mimics the structure of meta-test tasks. It is worth mentioning that meta-training tasks are sampled from the local base dataset $\mathcal{D}_b^{(i)}$ of each client, while meta-test tasks are sampled from the local novel dataset $\mathcal{D}_n^{(i)}$. That being said, the class set of samples in meta-training tasks is a subset of C_b, while the class set of samples in meta-test tasks is a subset of C_n, which is distinct from C_b. The main idea of federated few-shot learning is to preserve the consistency between meta-training and meta-test so that the model can learn meta-knowledge from clients for better generalization performance to novel classes C_n.

Specifically, to construct a meta-training task $\mathcal{T}$ in client $\mathbb{C}^{(i)}$ from its local base dataset $\mathcal{D}_b^{(i)}$, we first randomly sample N classes from $\mathcal{D}_b^{(i)}$. Then we randomly select K samples from each of the N classes (i.e., N-way K-shot) to establish the support set $\mathcal{S}$. Similarly, the query set $\mathcal{Q}$ consists of Q different samples (distinct from $\mathcal{S}$) from the same N classes. The components of the meta-training task $\mathcal{T}$ is formulated as follows:

$$\begin{aligned} \mathcal{S} &= \{(x_1, y_1), (x_2, y_2), \ldots, (x_{N\times K}, y_{N\times K})\}, \\ \mathcal{Q} &= \{(q_1, y_1'), (q_2, y_2'), \ldots, (q_Q, y_Q')\}, \\ \mathcal{T} &= \{\mathcal{S}, \mathcal{Q}\}, \end{aligned} \tag{1}$$

where x_i (or q_i) is a data sample in the sampled N classes, and y_i (or y_i') is the corresponding label. Note that during meta-test, each meta-test task shares a similar structure to meta-training tasks, except that the samples are from the local novel dataset $\mathcal{D}_n^{(i)}$, which are distinct from $\mathcal{D}_b^{(i)}$.

3 METHODOLOGY

In this part, we introduce the overall design of our proposed framework F^{2}L in detail. Specifically, we formulate the *federated few-shot learning* problem under the prevailing N-way K-shot learning framework. Our target of conducting federated few-shot learning is to learn meta-knowledge from a set of I clients $\{\mathbb{C}^{(i)}\}_{i=1}^{I}$ with different data distributions, and generalize such meta-knowledge to meta-test tasks. Nevertheless, it remains difficult to conduct federated few-shot learning due to the challenging issues of global data variance and local data insufficiency as mentioned before. Therefore, as illustrated in Fig 2, we propose a decoupled meta-learning framework to mitigate disruption from the servers. We further propose two update strategies to leverage global knowledge. The overview process is presented in Fig 3.

3.1 Decoupled Meta-Learning Framework

3.1.1 Federated Learning Framework. We consider a server-model, which consists of an encoder q_ϕ and a classifier f_ϕ that are shared among clients. We denote the overall model parameters in the server-model as ϕ. Specifically, $q_\phi : \mathbb{R}^d \rightarrow \mathbb{R}^k$ is a function that maps each sample into a low-dimensional vector $\mathbf{h}_\phi \in \mathbb{R}^k$, where d is the input feature dimension, and k is the dimension of learned representations. Taking the representation $\mathbf{h}_\phi$ as input, the classifier $f_\phi : \mathbb{R}^k \rightarrow C_b$ maps each $\mathbf{h}_\phi$ to the label space of base classes C_b and outputs the prediction $\mathbf{p}_\phi \in \mathbb{R}^{|C_b|}$, where each element in $\mathbf{p}_\phi$ denotes the classification probability regarding each class in C_b.

Following the prevalent FedAvg [36] strategy for FL, the training of server-model is conducted on all clients through T rounds. In each round t, the server $\mathbb{S}$ first sends the server-model parameters ϕ to all clients, and each client will conduct a local meta-training process on τ randomly sampled meta-training tasks. Then the server $\mathbb{S}$ will perform aggregation on parameters received from clients:

$$\phi^{t+1} = \frac{1}{I}\sum_{i=1}^{I} \widetilde{\phi}_i^t, \tag{2}$$

where $\widetilde{\phi}_i^t$ denotes the locally updated server-model parameters by client $\mathbb{C}^{(i)}$ on round t. ϕ^{t+1} denotes the aggregated server-model parameters which will be distributed to clients at the beginning of the next round. In this way, the server can learn a shared model for all clients in a federated manner.

Although the standard strategy of learning a single shared model for all clients achieves decent performance on general FL tasks [13, 36], it can be suboptimal for federated few-shot learning. Due to the global data variance among clients, the aggregated model on the server will disrupt the learning of meta-knowledge in each client [23]. As a result, the local learning of meta-knowledge in clients will become more difficult. In contrast, we propose to further introduce a client-model, which is uniquely learned and preserved by each client, to locally learn meta-knowledge. In other words, its model parameters will not be sent back to the server for aggregation. In this manner, we can separate the learning of client-model (meta-knowledge) and server-model (client-invariant knowledge) so that the learning of meta-knowledge is not disrupted.

Specifically, for client $\mathbb{C}^{(i)}$, the client-model also consists of an encoder q_{ψ_i} and a classifier f_{ψ_i}. We denote the overall model parameters in the client-model for client $\mathbb{C}^{(i)}$ as ψ_i. In particular, the encoder q_{ψ_i} takes the representation $\mathbf{h}_\phi$ learned by the encoder q_ϕ in server-model as input, and outputs a hidden representation $\mathbf{h}_\psi \in \mathbb{R}^k$. Such a design ensures that the client-model encoder q_{ψ_i} does not need to process the raw sample and thus can be a small model, which is important when clients only preserve limited computational resources [6]. Then the classifier f_{ψ_i} maps $\mathbf{h}_\psi$ to predictions $\mathbf{p}_\psi \in \mathbb{R}^N$ of the N classes.

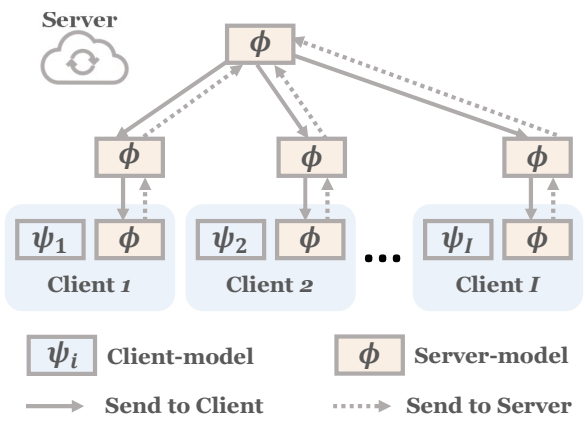


Figure 2: The illustration of our decoupled meta-learning framework. ψ denotes the client-model, which will be locally kept in each client. ϕ denotes the server-model, which will be aggregated and sent to the server.

3.1.2 Local Meta-training on Clients. Based on the episodic learning strategy, in each round, the training process of each client $\mathbb{C}^{(i)}$ is conducted through τ steps, where each step is a local update based on a meta-training task randomly sampled from the local base dataset $\mathcal{D}_b^{(i)}$. In particular, for client $\mathbb{C}^{(i)}$ on round $t = 1, 2, \ldots, T$ and step $s = 1, 2, \ldots, \tau$, we denote the sampled meta-task as $\mathcal{T}_i^{t,s} = \{\mathcal{S}_i^{t,s}, \mathcal{Q}_i^{t,s}\}$. To learn meta-knowledge from meta-task $\mathcal{T}_i^{t,s}$, we adopt the prevalent MAML [15] strategy to update client-model in one fine-tuning step and one meta-update step. We first fine-tune the client-model to fast adapt it to support set $\mathcal{S}_i^{t,s}$:

$$\widetilde{\psi}_i^{t,s} = \psi_i^{t,s} - \alpha_{ft} \nabla_\psi \mathcal{L}_{ft}\left(\mathcal{S}_i^{t,s}; \{\phi_i^{t,s}, \psi_i^{t,s}\}\right), \tag{3}$$

where $\mathcal{L}_{ft}$ is the fine-tuning loss, which is the cross-entropy loss calculated on the support set $\mathcal{S}_i^{t,s}$. Here, α_{ft} is the learning rate, and $\psi_i^{t,s}$ (or $\phi_i^{t,s}$) denotes the parameters of client-model (or server-model) on round t and step s. Then we update the client-model based on the query set $\mathcal{Q}_i^{t,s}$:

$$\psi_i^{t,s+1} = \psi_i^{t,s} - \alpha_\psi \nabla_\psi \mathcal{L}_\psi\left(\mathcal{Q}_i^{t,s}; \{\phi_i^{t,s}, \widetilde{\psi}_i^{t,s}\}\right), \tag{4}$$

where $\mathcal{L}_\psi$ is the loss for client-model on the query set $\mathcal{Q}_i^{t,s}$, and α_ψ is the meta-learning rate for ψ. In this regard, we can update client-model with our global-to-local knowledge distillation strategy. For the update of server-model, we conduct one step of update based on the support set and parameters of client-model:

$$\phi_i^{t,s+1} = \phi_i^{t,s} - \alpha_\phi \nabla_\phi \mathcal{L}_\phi\left(\mathcal{S}_i^{t,s}; \{\phi_i^{t,s}, \psi_i^{t,s}\}\right), \tag{5}$$

where $\mathcal{L}_\phi$ is the loss for the server-model, and α_ϕ is the meta-learning rate for ϕ. In this manner, we can update the server-model with our local-to-global knowledge transfer strategy. After repeating the above updates for τ steps, the final parameters of server-model $\phi_i^{t,\tau}$ is used as $\widetilde{\phi}_i^t$ in Eq. (2) and sent back to the server for aggregation, while the client-model (with parameters $\psi_i^{t,\tau}$) will be kept locally. By doing this, we can decouple the learning of local meta-knowledge in client-model while learning client-invariant knowledge in server-model to avoid disruption from the server.

3.2 Local-to-Global Knowledge Transfer

With our decoupled meta-learning framework, we can mitigate the disruption to the learning of local meta-knowledge in each client. Nevertheless, we still need to transfer the learned meta-knowledge to server-model (i.e., Local-to-global Knowledge Transfer), so that it can be further leveraged by other clients to handle the local data insufficiency issue. Specifically, to effectively transfer local meta-knowledge, we propose to maximize the mutual information between representations learned from server-model encoder q_ϕ and client-model encoder q_ψ. In this way, the server-model can maximally absorb the information in the learned local meta-knowledge.

3.2.1 Mutual Information Maximization. Given a meta-training task $\mathcal{T} = \{\mathcal{S}, \mathcal{Q}\}$, as described in Sec. 3.1, the server-model encoder q_ϕ and client-model encoder q_ψ will output $\mathbf{h}_\phi$ and $\mathbf{h}_\psi$ for each sample, respectively. By stacking the learned representations of samples in the support set $\mathcal{S}$ ($|\mathcal{S}| = D$, where $D = N \times K$), we can obtain the representations of support samples learned by the server-model, i.e., $\mathbf{H}_\phi \in \mathbb{R}^{D\times k}$, and the client-model, i.e., $\mathbf{H}_\psi \in \mathbb{R}^{D\times k}$. For simplicity, we omit the annotations of round t, step s, and client i. The objective of maximizing the information between $\mathbf{H}_\phi$ and $\mathbf{H}_\psi$ can be formally represented as follows:

$$\max_\phi I(\mathbf{H}_\phi; \mathbf{H}_\psi) = \max_\phi \sum_{i=1}^{D}\sum_{j=1}^{D} p(\mathbf{h}_\phi^i, \mathbf{h}_\psi^j; \phi) \log \frac{p(\mathbf{h}_\psi^j|\mathbf{h}_\phi^i; \phi)}{p(\mathbf{h}_\phi^j; \phi)}, \tag{6}$$

where $\mathbf{h}_\phi^i$ (or $\mathbf{h}_\psi^i$) is the i-th row of $\mathbf{H}_\phi$ (or $\mathbf{H}_\psi$). Since the mutual information $I(\mathbf{H}_\phi; \mathbf{H}_\psi)$ is difficult to obtain and thus infeasible to be maximized [39], we re-write it to achieve a more feasible form:

$$I(\mathbf{H}_\phi; \mathbf{H}_\psi) = \sum_{i=1}^{D}\sum_{j=1}^{D} p(\mathbf{h}_\phi^i|\mathbf{h}_\psi^j; \phi) p(\mathbf{h}_\psi^j; \phi) \log \frac{p(\mathbf{h}_\psi^j|\mathbf{h}_\phi^i; \phi)}{p(\mathbf{h}_\psi^j; \phi)}. \tag{7}$$

Since the support set $\mathcal{S}$ of size D is randomly sampled, we can assume that the prior probability $p(\mathbf{h}_\psi^j; \phi)$ follows a uniform distribution, and set it as $p(\mathbf{h}_\psi^j; \phi) = 1/D$. According to the Bayes' theorem, the Eq. (7) becomes:

$$I(\mathbf{H}_\phi; \mathbf{H}_\psi) = \frac{1}{D}\sum_{i=1}^{D}\sum_{j=1}^{D} p(\mathbf{h}_\phi^i|\mathbf{h}_\psi^j; \phi)\left(\log(p(\mathbf{h}_\psi^j|\mathbf{h}_\phi^i; \phi)) + \log D\right). \tag{8}$$

We next present alternative strategies to estimate $p(\mathbf{h}_\phi^i|\mathbf{h}_\psi^j; \phi)$ and $p(\mathbf{h}_\psi^j|\mathbf{h}_\phi^i; \phi)$ in detail.

3.2.2 Estimation of $p(\mathbf{h}_\phi^i|\mathbf{h}_\psi^j; \phi)$. Since the client-model is fine-tuned on the support set $\mathcal{S}$ of the meta-task $\mathcal{T}$, we can leverage the classification results of the client-model to estimate $p(\mathbf{h}_\phi^i|\mathbf{h}_\psi^j; \phi)$. We denote $C(j)$ as the set of sample indices in the support set $\mathcal{S}$ that shares the same class as the j-th sample (including itself), i.e., $C(j) \equiv \{k : y_k = y_j, k = 1, 2, \ldots, D\}$. Here, we first set $p(\mathbf{h}_\phi^i|\mathbf{h}_\psi^j; \phi) = 0$ for all $i \notin C(j)$, since we assume the client-model can only infer representations from the same class. Intuitively, in the

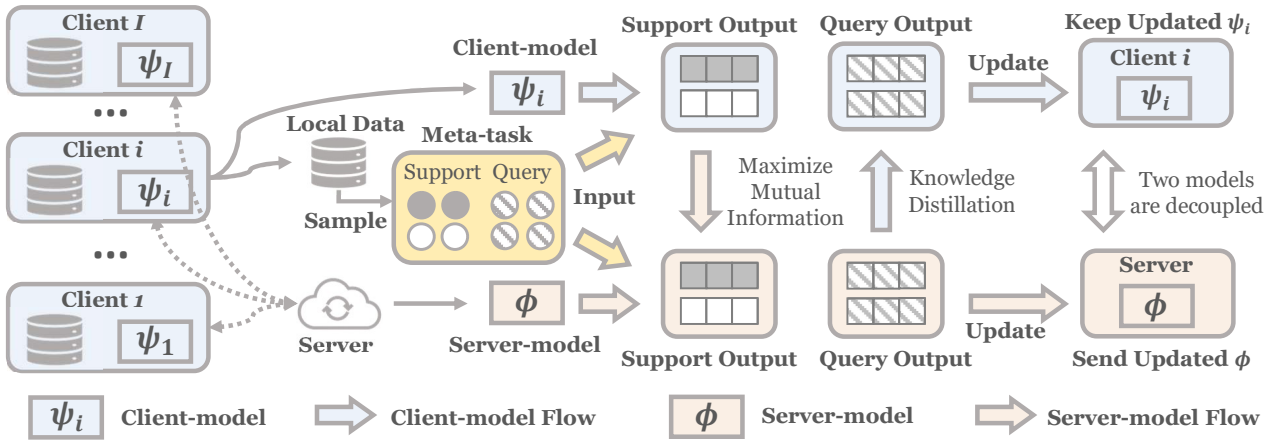


Figure 3: An illustration of the overall process of our framework F^{2}L. Specifically, each client receives the server-model from the server at the beginning of each round. To perform one step of local update, each client first samples a meta-task (2-way 2-shot in the illustration), which consists of a support set and a query set, from the local data. Then the server-model and the client-model will both compute output for the support samples and query samples. After that, the server-model and the client-model are updated via mutual information maximization and knowledge distillation, respectively. Finally, the server-model is sent back to the server for aggregation, while the client-model is locally preserved by each client.

case of $i \in C(j)$, which means the i-th and the j-th samples share the same class, $p(\mathbf{h}^i_\phi|\mathbf{h}^j_\psi;\phi)$ can be considered as the confidence of client-model regarding the class of the j-the sample. Therefore, it should reflect the degree to which the sample representation $\mathbf{h}^j_\psi$ is relevant to its class. Utilizing the client-model classification output (i.e., normalized class probabilities) for the i-th sample $\mathbf{p}^i_\psi \in \mathbb{R}^N$, we can compute $p(\mathbf{h}^i_\phi|\mathbf{h}^j_\psi;\phi)$ as follows:

$$p(\mathbf{h}^i_\phi|\mathbf{h}^j_\psi;\phi) = \begin{cases} \dfrac{\mathbf{p}^i_\psi(y_j)}{\sum_{k\in C(j)} \mathbf{p}^k_\psi(y_j)} & \text{if } i \in C(j) \\ 0 & \text{otherwise} \end{cases}, \tag{9}$$

where $\mathbf{p}^i_\psi(y_j) \in \mathbb{R}$ denotes the classification probability for the i-th sample regarding class y_j ($y_i = y_j$ when $i \in C(j)$).

3.2.3 Estimation of $p(\mathbf{h}^j_\psi|\mathbf{h}^i_\phi;\phi)$. Next we elaborate on how to estimate $p(\mathbf{h}^j_\psi|\mathbf{h}^i_\phi;\phi)$. Although we can similarly leverage the classification results of the server-model, such a strategy lacks generalizability. This is because the server-model aims at classifying all base classes instead of the N classes in each meta-training task. We instead propose to estimate $p(\mathbf{h}^j_\psi|\mathbf{h}^i_\phi;\phi)$ based on the Euclidean distance (divided by 2 for simplicity) between learned representations of the server-model and the client-model. Specifically, we normalize the distances with a softmax function:

$$p(\mathbf{h}^j_\psi|\mathbf{h}^i_\phi;\phi) = \frac{\exp\left(-\|\mathbf{h}^i_\phi - \mathbf{h}^j_\psi\|_2^2/2\right)}{\sum_{k\in C(i)} \exp\left(-\|\mathbf{h}^i_\phi - \mathbf{h}^k_\psi\|_2^2/2\right)}. \tag{10}$$

Then if we further apply the ℓ_2 normalization to both $\mathbf{h}^i_\phi$ and $\mathbf{h}^j_\psi$, we can obtain $\|\mathbf{h}^i_\phi - \mathbf{h}^j_\psi\|_2^2/2 = 1 - \mathbf{h}^i_\phi \cdot \mathbf{h}^j_\psi$. Moreover, since the value of $\sum_{i=1}^D \sum_{j=1}^D p(\mathbf{h}^i_\phi|\mathbf{h}^j_\psi;\phi)$ equals a constant D, the term $\sum_{i=1}^D \sum_{j=1}^D p(\mathbf{h}^i_\phi|\mathbf{h}^j_\psi;\phi) \cdot \log(D)/D$ in Eq. (8) is also a constant and thus can be ignored in the objective:

$$\frac{1}{D}\sum_{i=1}^D\sum_{j=1}^D p(\mathbf{h}^i_\phi|\mathbf{h}^j_\psi;\phi)\log(D) = \frac{1}{D}\cdot D\cdot\log(D) = \log(D). \tag{11}$$

Combining the above equations, the optimal server-model parameter ϕ^* for the final optimization objective (i.e., $\max_\phi I(\mathbf{H}_\phi;\mathbf{H}_\psi)$) can be obtained as follows:

$$\phi^* = \operatorname*{argmax}_\phi I(\mathbf{H}_\phi;\mathbf{H}_\psi) = \operatorname*{argmin}_\phi \mathcal{L}_{MI}. \tag{12}$$

Here $\mathcal{L}_{MI}$ is defined as follows:

$$\begin{aligned}\mathcal{L}_{MI} &= \frac{1}{D}\sum_{i=1}^D\sum_{j=1}^D p(\mathbf{h}^i_\phi|\mathbf{h}^j_\psi;\phi)\log(p(\mathbf{h}^j_\psi|\mathbf{h}^i_\phi;\phi)) \\ &= \frac{1}{D}\sum_{j=1}^D\sum_{i\in C(j)} -\frac{\mathbf{p}^i_\psi(y_j)\left(\mathbf{h}^i_\phi\cdot\mathbf{h}^j_\psi\right)}{\sum_{k\in C(j)}\mathbf{p}^k_\psi(y_j)} \\ &\quad + \frac{\mathbf{p}^i_\psi(y_j)}{\sum_{k\in C(j)}\mathbf{p}^k_\psi(y_j)}\log\left(\sum_{k\in C(i)}\exp\left(\mathbf{h}^i_\phi\cdot\mathbf{h}^k_\psi\right)\right),\end{aligned} \tag{13}$$

where we exchange the order of summation over i and j for clarity. It is noteworthy that $\mathcal{L}_{MI}$ is different from the InfoNCE loss [18, 39], which considers different augmentations of samples, while $\mathcal{L}_{MI}$ focuses on the classes of samples in $\mathcal{S}$. Moreover, $\mathcal{L}_{MI}$ also differs from the supervised contrastive loss [25], which combines various augmentations of samples and label information. In contrast, our loss targets at transferring the meta-knowledge by maximally preserving the mutual information between representations learned by the server-model and the client-model. More differently, the term $\mathbf{p}^i_\psi(y_j)/\sum_{k\in C(j)}\mathbf{p}^k_\psi(y_j)$ acts as an adjustable weight that measures the importance of a sample to its class. Combining the objective

described in Eq. (13) and the standard cross-entropy loss, we can obtain the final loss for the server-model:

$$\mathcal{L}_{\phi} = (1 - \lambda_{MI})\mathcal{L}_{CE}(\mathcal{S}) + \lambda_{MI}\mathcal{L}_{MI}, \tag{14}$$

where $\mathcal{L}_{CE}(\mathcal{S})$ is defined as follows:

$$\mathcal{L}_{CE}(\mathcal{S}) = -\frac{1}{D}\sum_{i=1}^{D}\sum_{j=1}^{|C_b|} y^i_{c_j} \log \mathbf{p}^i_{\phi}(c_j), \tag{15}$$

where $\mathbf{p}^i_{\phi}(c_j) \in \mathbb{R}$ denotes the classification probability for the i-th support sample belonging to the j-th class c_j in C_b, computed by the server-model. Here $y^i_{c_j} = 1$ if the i-th support sample belongs to c_j, and $y^i_{c_j} = 0$, otherwise. Moreover, $\lambda_{MI} \in [0, 1]$ is an adjustable hyper-parameter to control the weight of $\mathcal{L}_{MI}$.

3.3 Global-to-Local Knowledge Distillation

With the learned meta-knowledge in each client transferred from the client-model to the server-model, other clients can leverage such meta-knowledge to deal with the local data insufficiency issue. However, since each meta-task only contains N classes, directly extracting meta-knowledge in the server-model can inevitably involve meta-knowledge from other classes, which can be harmful to the learning of local meta-knowledge from these N classes in each client. Instead, we propose a partial knowledge distillation strategy to selectively extract useful knowledge from the server-model, i.e., global-to-local knowledge distillation.

3.3.1 Partial Knowledge Distillation. Specifically, we focus on the output classification probabilities of the server-model regarding the N classes in support set $\mathcal{S}$ while ignoring other classes. In this regard, we can extract the information that is crucial for learning local meta-knowledge from these N classes and also reduce the irrelevant information from other classes.

Particularly, we consider the same meta-task $\mathcal{T} = \{\mathcal{S}, \mathcal{Q}\}$. We denote the output probabilities for the i-th query sample q_i in $\mathcal{Q}$ (with label y_i) of the server-model and the client-model as $\mathbf{p}^i_{\phi} \in \mathbb{R}^{|C_b|}$ and $\mathbf{p}^i_{\psi} \in \mathbb{R}^N$, respectively. It is noteworthy that the N classes in this meta-task, denoted as C_m, are sampled from the base classes C_b (i.e., $|C_m| = N$ and $C_m \subset C_b$). Therefore, the output of server-model (i.e., $\mathbf{p}^i_{\phi}$) will include the probabilities of classes in C_m. In particular, we enforce the probabilities of in C_m from the client-model to be consistent with the probabilities of the same classes from the server-model. As a result, the learning of local meta-knowledge can leverage the information of data in the same N classes from other clients, which is encoded in the server-model. In this regard, we can handle the local data insufficiency issue by involving information from other clients while reducing the irrelevant information from other classes not in C_m. In particular, by utilizing the output of the server-model as the soft target for the client-model, we can achieve an objective as follows:

$$\mathcal{L}_{KD} = -\frac{1}{Q}\sum_{i=1}^{Q}\sum_{j=1}^{N} \mathbf{q}^i_{\phi}(c_j) \log \mathbf{q}^i_{\psi}(c_j), \tag{16}$$

where c_j is the j-th class in C_m (i.e., the N classes in meta-task $\mathcal{T}$). $\mathbf{q}^i_{\phi}(c_j)$ and $\mathbf{q}^i_{\psi}(c_j)$ are the knowledge distillation values for c_j from server-model and client-model, respectively. Specifically, the values of $\mathbf{q}^i_{\phi}(c_j)$ and $\mathbf{q}^i_{\psi}(c_j)$ are obtained via the softmax normalization:

$$\mathbf{q}^i_{\phi}(c_j) = \frac{\exp(\mathbf{z}^i_{\phi}(c_j)/T_i)}{\sum_{k=1}^{N}\exp(\mathbf{z}^i_{\phi}(c_k)/T_i)}, \tag{17}$$

$$\mathbf{q}^i_{\psi}(c_j) = \frac{\exp(\mathbf{z}^i_{\psi}(c_j)/T_i)}{\sum_{k=1}^{N}\exp(\mathbf{z}^i_{\psi}(c_k)/T_i)}, \tag{18}$$

where $\mathbf{z}^i_{\phi}(c_j)$ are $\mathbf{z}^i_{\psi}(c_j))$ are the logits (i.e., output before softmax normalization) of class c_j from server-model and client-model, respectively. T_i is the temperature parameter for the i-th query sample. In this way, we can ensure that $\sum_{j=1}^{N}\mathbf{q}^i_{\phi}(c_j) = \sum_{j=1}^{N}\mathbf{q}^i_{\psi}(c_j) = 1$.

3.3.2 Adaptive Temperature Parameter. Generally, a larger value of T_i denotes that the client-model focuses more on extracting information from the other classes in C_m [19] (i.e., $\{c|c \in C_m, c \neq y_i\}$), denoted as negative classes. Since the classification results can be erroneous in the server-model, we should adaptively adjust the value of T_i for each meta-task to reduce the adverse impact of extracting misleading information from the server-model. However, although negative classes can inherit useful information for classification, such information is generally noisier when the output probabilities of these negative classes are smaller. Therefore, to estimate the importance degree of each negative class, we consider the maximum output logit for negative classes to reduce potential noise. Particularly, if the probability of a negative class from the server-model is significantly larger than other classes, we can conjecture that this class is similar to y_i and thus potentially contains the crucial information to distinguish them. Specifically, the temperature parameter T_i for the i-th query sample q_i is computed as follows:

$$T_i = \sigma\left(\frac{\max_{c \in C_m, c \neq y_i}\exp(\mathbf{z}^i_{\phi}(c))}{\exp(\mathbf{z}^i_{\phi}(y_i))}\right), \tag{19}$$

where $\sigma(\cdot)$ denotes the Sigmoid function, and y_i is the label of q_i. In this way, the temperature parameter T_i will increase when the ratio between the largest probability in negative classes and the probability for y_i is larger. As a result, the client-model will focus more on the negative class information. Then by further incorporating the cross-entropy loss on the query set $\mathcal{Q}$, we can obtain the final loss for the client-model:

$$\mathcal{L}_{\psi} = (1 - \lambda_{KD})\mathcal{L}_{CE}(\mathcal{Q}) + \lambda_{KD}\mathcal{L}_{KD}, \tag{20}$$

where $\mathcal{L}_{CE}(\mathcal{Q})$ is defined as follows:

$$\mathcal{L}_{CE}(\mathcal{Q}) = -\frac{1}{Q}\sum_{i=1}^{Q}\sum_{j=1}^{N} y^i_{c_j} \log \mathbf{p}^i_{\psi}(c_j), \tag{21}$$

where $\mathbf{p}^i_{\psi}(c_j)$ is the probability of the i-th query sample belonging to class c_j computed by the client-model. $y^i_{c_j} = 1$ if the i-th query sample belongs to c_j, and $y^i_{c_j} = 0$, otherwise. Moreover, $\lambda_{KD} \in [0, 1]$ is an adjustable hyper-parameter to control the weight of $\mathcal{L}_{KD}$. In this manner, the client-model can selectively learn useful knowledge from both the local and global perspectives, i.e., global-to-local knowledge distillation.

3.4 Overall Learning Process

With the proposed losses $\mathcal{L}_\phi$ and $\mathcal{L}_\psi$, on each round, we can conduct meta-training on each client $\mathbb{C}^{(i)}$ by sampling τ meta-training tasks from the local base dataset $\mathcal{D}_b^{(i)}$. The detailed process is described in Algorithm 1. After T rounds of meta-training on all the clients, we have obtained a model that accommodates comprehensive meta-knowledge for federated few-shot learning. For the meta-test phase, since we have aggregated learned local meta-knowledge from each client to the server-model, we can leverage the server-model to generate data representations for classification. Specifically, during evaluation, for each meta-test task $\mathcal{T} = \{\mathcal{S}, \mathcal{Q}\}$ sampled from local novel datasets $\{\mathcal{D}_n^{(i)}\}_{i=1}^I$ in all clients, we follow the same process as meta-training including fine-tuning, except that the meta-update process is omitted. The output of the client-model will be used for classification.

4 EXPERIMENTS

In this part, we conduct extensive experiments to evaluate our framework F^{2}L on four few-shot classification datasets covering both news articles and images under the federated scenario.

4.1 Datasets

In this section, we introduce four prevalent real-world datasets used in our experiments, covering both news articles and images: **20 Newsgroup** [28], **Huffpost** [38], **FC100** [40], and **miniImageNet** [52]. In particular, 20 Newsgroup and Huffpost are online news article datasets, while FC100 and miniImageNet are image datasets. The details are as follows:

- **20 Newsgroup** [28] is a text dataset that consists of informal discourse from news discussion forums. There are 20 classes for documents in this dataset, where each class belongs to one of six top-level categories. The classes are split as 8/5/7 for training/validation/test, respectively.
- **Huffpost** [38] is a text dataset containing news headlines published on HuffPost[2] between 2012 and 2018. Generally, the headlines are significantly shorter and less grammatical than the 20 Newsgroup dataset. Moreover, each headline belongs to one of 41 classes, which are then split as 20/5/16 for training/validation/test, respectively.
- **FC100** [40] is an image classification dataset based on CIFAR-100 [27]. Specifically, this dataset contains 100 image classes, where each class maintains 600 images with a low 32×32 resolution. The classes are split as 60/20/20 for training/validation/test, respectively.
- **miniImageNet** [52] is an image dataset extracted from the full ImageNet dataset [8]. This dataset consists of 100 image classes, and each class maintains 600 images with a resolution of 84×84. The classes are split as 64/16/20 for training/validation/test, respectively.

4.2 Experimental Settings

To validate the performance of our framework F^{2}L, we conduct experiments with the following baselines for a fair comparison:

- *Local*. This baseline is non-distributed, which means we train an individual model for each client on the local data. The meta-test process is conducted on all meta-test tasks, and the averaged results of all models are reported.
- *FL-MAML*. This baseline leverages the MAML [15] strategy to perform meta-learning on each client. The updated model parameters will be sent back to the server for aggregation.
- *FL-Proto*. This baseline uses ProtoNet [44] as the model in each client. The classification is based on the Euclidean distances between query samples and support samples.
- *FedFSL* [14]. This method combines MAML and an adversarial learning strategy [17, 43] to construct a consistent feature space. The aggregation is based on FedAvg [36].

During meta-training, we perform updates for the client-model and the server-model according to Algorithm 1. Finally, the server-model that achieves the best result on validation will be used for meta-test. Then during meta-test, we evaluate the server-model on a series of 100 randomly sampled meta-test tasks from local novel datasets $\{\mathcal{D}_n^{(i)}\}_{i=1}^I$ in all clients. For consistency, the class split of C_b and C_n is identical for all baseline methods. The classification accuracy over these meta-test tasks will be averaged as the final results. The specific parameter settings are provided in Appendix C.3. For the specific choices for the encoder and classifier in server-model and client-model (i.e., q_ϕ, f_ϕ, q_ψ, and f_ψ) and model parameters, we provide further details in Appendix C.1. Note that for a fair comparison, we utilize the same encoder for all methods.

4.3 Overall Evaluation Results

We present the overall performance comparison of our framework and baselines on federated few-shot learning in Table 1. Specifically, we conduct experiments under two few-shot settings: 5-way 1-shot and 5-way 5-shot. Moreover, to demonstrate the robustness of our framework under different data distributions, we partition the data in both IID and non-IID settings. For the IID partition, the samples of each class are uniformly distributed to all clients. For non-IID partition, we follow the prevailing strategy [21, 60] and distribute samples to all clients based on the Dirichlet distribution with its concentration parameter set as 1.0. The evaluation metric is the average classification accuracy over ten repetitions. From the overall results, we can obtain the following observations:

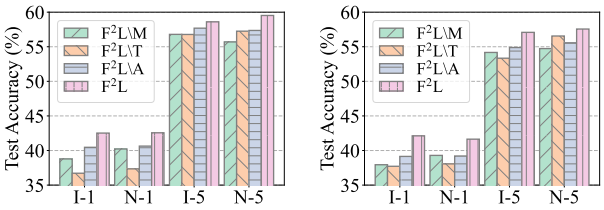
- Our framework F^{2}L outperforms all other baselines on various news article and image datasets under different few-shot settings (1-shot and 5-shot) and data distributions (IID and non-IID). The results validate the effectiveness of our framework on federated few-shot learning.
- Conventional few-shot methods such as Prototypical Network [44] and MAML [15] exhibit similar performance compared with the Local baseline. The result demonstrates that directly applying few-shot methods to federated learning brings less competitive improvements over local training. This is because such methods are not proposed for federated learning and thus lead to unsatisfactory training performance under the federated setting.
- The performance of all methods degrades at different extents when the data distribution is changed from IID to non-IID. The main reason is that the variety of classes in each client

[2] https://www.huffpost.com/

Table 1: The overall federated few-shot learning results of various models on four datasets under IID and Non-IID settings (5-way), where accuracy and standard deviation are reported in %. The best results are presented as bold.

Dataset	20 Newsgroup				Huffpost			
Distribution	IID		Non-IID		IID		Non-IID	
Setting	1-shot	5-shot	1-shot	5-shot	1-shot	5-shot	1-shot	5-shot
Local	31.53 ± 1.68	42.73 ± 1.51	29.64 ± 1.81	41.01 ± 2.40	34.02 ± 1.67	49.95 ± 1.54	33.09 ± 2.28	47.18 ± 1.43
FL-MAML	32.89 ± 1.86	44.34 ± 1.66	31.60 ± 1.44	43.84 ± 1.97	37.47 ± 1.43	52.85 ± 1.43	36.01 ± 2.17	50.56 ± 2.08
FL-Proto	35.62 ± 2.07	46.04 ± 1.92	32.79 ± 1.41	43.82 ± 1.85	37.87 ± 1.23	51.90 ± 1.43	34.05 ± 1.35	50.52 ± 1.33
FedFSL	36.56 ± 1.41	46.37 ± 1.82	35.84 ± 1.49	45.89 ± 1.72	39.18 ± 1.42	53.81 ± 1.36	37.86 ± 1.46	52.18 ± 1.82
F^2L	**39.80 ± 1.80**	**49.64 ± 1.32**	**39.00 ± 1.36**	**49.44 ± 1.98**	**42.12 ± 2.12**	**57.88 ± 2.17**	**41.64 ± 1.81**	**57.12 ± 1.87**

Dataset	FC100				miniImageNet			
Distribution	IID		Non-IID		IID		Non-IID	
Setting	1-shot	5-shot	1-shot	5-shot	1-shot	5-shot	1-shot	5-shot
Local	33.45 ± 1.68	50.89 ± 1.56	32.40 ± 1.76	50.29 ± 2.24	47.82 ± 1.68	64.30 ± 1.59	46.81 ± 2.03	64.06 ± 1.45
FL-MAML	34.10 ± 1.29	50.66 ± 1.68	36.06 ± 1.78	50.35 ± 1.57	49.74 ± 1.40	65.55 ± 1.57	47.64 ± 1.36	63.56 ± 1.13
FL-Proto	36.11 ± 1.49	54.74 ± 2.05	35.54 ± 1.71	52.31 ± 1.76	51.32 ± 1.41	66.67 ± 2.06	50.82 ± 1.82	65.09 ± 1.90
FedFSL	39.38 ± 1.95	52.25 ± 1.84	38.60 ± 2.00	53.90 ± 1.80	55.75 ± 2.06	70.59 ± 1.97	53.52 ± 2.01	69.56 ± 1.86
F^2L	**42.52 ± 2.06**	**58.60 ± 2.09**	**42.56 ± 2.25**	**59.52 ± 2.14**	**56.72 ± 1.79**	**74.23 ± 2.32**	**56.16 ± 2.05**	**73.24 ± 2.02**



(a) FC100 **(b) Huffpost**

Figure 4: Ablation study of our framework on FC100 and Huffpost. I-K (or N-K) denotes the setting of 5-way K-shot under IID (or non-IID) distributions. M denotes the decoupled framework, T means the local-to-global knowledge transfer, and A demotes the global-to-local knowledge distillation.

results in a more complex class distribution and brings difficulties to the classification task. Nevertheless, by effectively transferring the meta-knowledge among clients, our framework is capable of alleviating such a problem under the non-IID scenario.

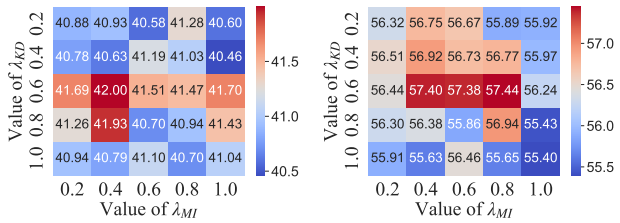
- When increasing the value of K (i.e., more support samples in each class), all methods achieve considerable performance gains. In particular, our framework F^2L obtains better results compared to other baselines, due to our decoupled meta-learning framework, which promotes the learning of meta-knowledge in the support samples.

4.4 Ablation Study

In this part, we conduct an ablation study on FC100 and Huffpost to validate the effectiveness of three crucial designs in F^2L (similar results observed in other datasets). First, we remove the decoupled strategy so that the client-model will also be sent to the server for aggregation. We refer to this variant as $F^2L\backslash M$. Second, we remove the local-to-global knowledge transfer module so that the meta-knowledge in the client-model will be effectively transferred to the server-model. This variant is referred to as $F^2L\backslash T$. Third, we eliminate the global-to-local knowledge distillation loss. In this way, the client-model cannot leverage the global knowledge in the server-model for learning meta-knowledge. We refer to this variant as $F^2L\backslash A$. The overall ablation study results are presented in Fig. 4. From the results, we observe that F^2L outperforms all variants, which verifies the effectiveness of the three designs in F^2L. Specifically, removing the design of local-to-global knowledge transfer leads to significant performance degradation. This result demonstrates that such a design can effectively aggregate learned meta-knowledge among clients and thus bring performance improvements. More significantly, without our decoupled strategy, the performance deteriorates rapidly when federated few-shot learning is conducted in the non-IID scenario. This phenomenon verifies the importance of mitigating the disruption from the server in the presence of complex data distributions among clients.

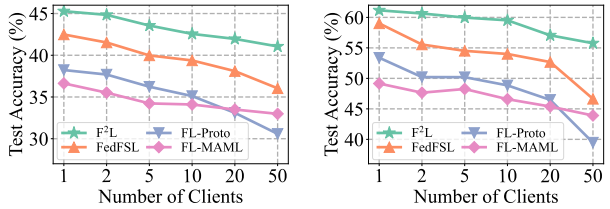
4.5 Parameter Sensitivity Study

4.5.1 Effect of λ_{MI} and λ_{KD}. In this section, we further conduct experiments to study the sensitivity of several parameters in our framework F^2L. During the process of transferring and achieving meta-knowledge, we introduce two novel losses $\mathcal{L}_{MI}$ and $\mathcal{L}_{KD}$, respectively, along with the traditional cross-entropy loss. To empirically evaluate the impact brought by different values of λ_{MI} and λ_{KD} in Eq. (14) and Eq. (20) , we adjust the values of λ_{MI} and λ_{KD} from 0 to 1 and present the results in Fig 5. From the results, we can observe that the performance generally increases with a



(a) 1-shot (b) 5-shot

Figure 5: The results with different values of λ_{MI} and λ_{KD} on Huffpost under the non-IID setting.



(a) 1-shot (b) 5-shot

Figure 6: The results of non-IID federated 1-shot and 5-shot learning on FC100 regarding the number of clients.

larger value of λ_{MI}, while decreasing with λ_{MI} approaches 1. The results indicate the importance of transferring learned local meta-knowledge, while also demonstrating that the cross-entropy loss is necessary. On the other hand, the performance first increases and then degrades when a larger value of λ_{KD} is presented. That being said, although partial knowledge distillation can enable each client to benefit from the global data, a larger λ_{KD} can potentially lead to more irrelevant information when learning local meta-knowledge.

4.5.2 Effect of Client Number. In this section, we study the robustness of our framework under the scenario with a varying number of clients. In particular, we keep the total training data unchanged, which means with more clients participating in the training process, each client preserves fewer training samples. As a result, the training performance will be inevitably reduced. Specifically, we partition the total training data into $I = 1, 2, 5, 10, 20$, and 50 clients. Note that $I = 1$ denotes the setting of completely centralized training. The results on FC100 with 1-shot and 5-shot settings are presented in Fig 6 (we have similar results for other datasets and omit them for brevity). From the results, we can observe that all methods encounter a performance drop in the presence of more clients. Nevertheless, our framework F^2L can reduce the adverse impact brought by more clients through effectively leveraging the global knowledge learned from all clients. In consequence, the performance degradation is less significant for F^2L.

5 RELATED WORK

5.1 Few-shot Learning

The objective of Few-shot Learning (FSL) is to learn transferable meta-knowledge from tasks with abundant information and generalize such knowledge to novel tasks that consist of only limited labeled samples [9, 11, 45, 49, 56]. Existing few-shot learning works can be divided into two categories: *metric-based* methods and *optimization-based* methods. The metric-based methods target at learning generalizable metric functions to classify query samples by matching them with support samples [35, 46, 54]. For instance, Prototypical Networks [44] learn a prototype representation for each class and conduct predictions based on the Euclidean distances between query samples and the prototypes. Relation Networks [46] learn relation scores for classification in a non-linear manner. On the other hand, optimization-based approaches generally optimize model parameters based on the gradients calculated from few-shot samples [24, 37, 41, 53]. As an example, MAML [15] proposes to optimize model parameters based on gradients on support samples to achieve fast generalization. In addition, LSTM-based meta-learner [41] adjusts the step size to adaptively update parameters during meta-training.

5.2 Federated Learning

Federated Learning (FL) enables multiple clients to collaboratively train a model without exchanging the local data explicitly [16, 22, 30, 47, 58, 63]. As a classic example, FedAvg [36] performs stochastic gradient descent (SGD) on each client to update model parameters and send them to the server. The server averages the received model parameters to achieve a global model for the next round. FedProx [32] incorporates a proximal term into the local update of each client to reduce the distance between the global model and the local model. To deal with the non-IID problem in FL, recent works also focus on personalization in FL [1, 3, 13, 48]. For instance, FedMeta [6] incorporates MAML [15] into the local update process in each client for personalization. FedRep [7] learns shared representations among clients. Moreover, FedFSL [14] proposes to combine MAML and an adversarial learning strategy [17, 43] to learn a consistent feature space.

6 CONCLUSION

In this paper, we study the problem of federated few-shot learning, which aims at learning a federated model that can achieve satisfactory performance on new tasks with limited labeled samples. Nevertheless, it remains difficult to perform federated few-shot learning due to two challenges: global data variance and local data insufficiency. To tackle these challenges, we propose a novel federated few-shot learning framework F^2L. In particular, we handle global data variance by decoupling the learning of local meta-knowledge. Then we leverage the global knowledge that is learned from all clients to tackle the local data insufficiency issue. We conduct extensive experiments on four prevalent few-shot learning datasets under the federated setting, covering both news articles and images. The experimental results further validate the superiority of our framework F^2L over other state-of-the-art baselines.

7 ACKNOWLEDGEMENTS

The work in this paper is supported by the National Science Foundation under grants (IIS-2006844, IIS-2144209, IIS-2223769, CNS-2154962, and BCS-2228534), the Commonwealth Cyber Initiative awards (VV-1Q23- 007 and HV-2Q23-003), the JP Morgan Chase Faculty Research Award, the Cisco Faculty Research Award, the Jefferson Lab subcontract 23-D0163, and the UVA 4-VA collaborative research grant.

REFERENCES

[1] Manoj Ghuhan Arivazhagan, Vinay Aggarwal, Aaditya Kumar Singh, and Sunav Choudhary. 2019. Federated learning with personalization layers. *arXiv:1912.00818* (2019).

[2] Christopher Briggs, Zhong Fan, and Peter Andras. 2020. Federated learning with hierarchical clustering of local updates to improve training on non-IID data. In *IJCNN*.

[3] Duc Bui, Kshitiz Malik, Jack Goetz, Honglei Liu, Seungwhan Moon, Anuj Kumar, and Kang G Shin. 2019. Federated user representation learning. *arXiv:1909.12535* (2019).

[4] Soumen Chakrabarti. 2002. *Mining the Web: Discovering knowledge from hypertext data*. Morgan Kaufmann.

[5] Michael Chau and Hsinchun Chen. 2008. A machine learning approach to web page filtering using content and structure analysis. *Decision Support Systems* 44, 2 (2008), 482–494.

[6] Fei Chen, Mi Luo, Zhenhua Dong, Zhenguo Li, and Xiuqiang He. 2018. Federated meta-learning with fast convergence and efficient communication. *arXiv:1802.07876* (2018).

[7] Liam Collins, Hamed Hassani, Aryan Mokhtari, and Sanjay Shakkottai. 2021. Exploiting shared representations for personalized federated learning. In *ICLR*.

[8] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *CVPR*.

[9] Kaize Ding, Jianling Wang, Jundong Li, Kai Shu, Chenghao Liu, and Huan Liu. 2020. Graph prototypical networks for few-shot learning on attributed networks. In *CIKM*.

[10] Kaize Ding, Qinghai Zhou, Hanghang Tong, and Huan Liu. 2021. Few-shot network anomaly detection via cross-network meta-learning. In *TheWebConf*.

[11] Simon S Du, Wei Hu, Sham M Kakade, Jason D Lee, and Qi Lei. 2020. Few-shot learning via learning the representation, provably. *arXiv:2002.09434* (2020).

[12] Moming Duan, Duo Liu, Xianzhang Chen, Renping Liu, Yujuan Tan, and Liang Liang. 2020. Self-balancing federated learning with global imbalanced data in mobile systems. *IEEE Transactions on Parallel and Distributed Systems* (2020).

[13] Alireza Fallah, Aryan Mokhtari, and Asuman Ozdaglar. 2020. Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach. In *NeurIPS*.

[14] Chenyou Fan and Jianwei Huang. 2021. Federated Few-Shot Learning with Adversarial Learning. In *WiOpt*.

[15] Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-agnostic meta-learning for fast adaptation of deep networks. In *ICML*.

[16] Xingbo Fu, Binchi Zhang, Yushun Dong, Chen Chen, and Jundong Li. 2022. Federated graph machine learning: A survey of concepts, techniques, and applications. *ACM SIGKDD Explorations Newsletter* (2022).

[17] Ian J Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron C Courville, and Yoshua Bengio. 2014. Generative Adversarial Nets. In *NIPS*.

[18] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. 2020. Momentum contrast for unsupervised visual representation learning. In *CVPR*.

[19] Geoffrey Hinton, Oriol Vinyals, Jeff Dean, et al. 2015. Distilling the knowledge in a neural network. *arXiv:1503.02531* 2, 7 (2015).

[20] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* (1997).

[21] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. 2019. Measuring the effects of non-identical data distribution for federated visual classification. *arXiv:1909.06335* (2019).

[22] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. 2021. Advances and open problems in federated learning. *Foundations and Trends® in Machine Learning* (2021).

[23] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. 2020. Scaffold: Stochastic controlled averaging for federated learning. In *ICML*.

[24] Mikhail Khodak, Maria-Florina F Balcan, and Ameet S Talwalkar. 2019. Adaptive gradient-based meta-learning methods. *NeurIPS* (2019).

[25] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. 2020. Supervised contrastive learning. In *NeurIPS*.

[26] Diederik P Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *ICLR*.

[27] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images. (2009).

[28] Ken Lang. 1995. Newsweeder: Learning to filter netnews. In *Machine Learning Proceedings 1995*.

[29] Kwonjoon Lee, Subhransu Maji, Avinash Ravichandran, and Stefano Soatto. 2019. Meta-learning with differentiable convex optimization. In *CVPR*.

[30] Daliang Li and Junpu Wang. 2019. Fedmd: Heterogenous federated learning via model distillation. *arXiv:1910.03581* (2019).

[31] Qinbin Li, Zeyi Wen, Zhaomin Wu, Sixu Hu, Naibo Wang, Yuan Li, Xu Liu, and Bingsheng He. 2021. A survey on federated learning systems: vision, hype and reality for data privacy and protection. *TKDE* (2021).

[32] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated optimization in heterogeneous networks. In *MLSys*.

[33] Wei Yang Bryan Lim, Nguyen Cong Luong, Dinh Thai Hoang, Yutao Jiao, Ying-Chang Liang, Qiang Yang, Dusit Niyato, and Chunyan Miao. 2020. Federated learning in mobile edge networks: A comprehensive survey. *IEEE Communications Surveys & Tutorials* (2020).

[34] Bing Liu. 2011. *Web data mining: exploring hyperlinks, contents, and usage data*. Vol. 1. Springer.

[35] Lu Liu, Tianyi Zhou, Guodong Long, Jing Jiang, and Chengqi Zhang. 2019. Learning to propagate for graph meta-learning. In *NeurIPS*.

[36] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *AISTATS*.

[37] Nikhil Mishra, Mostafa Rohaninejad, Xi Chen, and Pieter Abbeel. 2018. A Simple Neural Attentive Meta-Learner. In *ICLR*.

[38] Rishabh Misra. 2018. News category dataset. *DOI: DOI: https://doi. org/10.13140/RG* 2, 20331.18729 (2018).

[39] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. In *arXiv:1807.03748*.

[40] Boris Oreshkin, Pau Rodríguez López, and Alexandre Lacoste. 2018. Tadam: Task dependent adaptive metric for improved few-shot learning. In *NeurIPS*.

[41] Sachin Ravi and Hugo Larochelle. 2016. Optimization as a model for few-shot learning. In *ICLR*.

[42] Avinash Ravichandran, Rahul Bhotika, and Stefano Soatto. 2019. Few-shot learning with embedded class models and shot-free meta training. In *CVPR*.

[43] Kuniaki Saito, Kohei Watanabe, Yoshitaka Ushiku, and Tatsuya Harada. 2018. Maximum classifier discrepancy for unsupervised domain adaptation. In *CVPR*.

[44] Jake Snell, Kevin Swersky, and Richard Zemel. 2017. Prototypical networks for few-shot learning. In *NeurIPS*.

[45] Qianru Sun, Yaoyao Liu, Tat-Seng Chua, and Bernt Schiele. 2019. Meta-transfer learning for few-shot learning. In *CVPR*.

[46] Flood Sung, Yongxin Yang, Li Zhang, Tao Xiang, Philip HS Torr, and Timothy M Hospedales. 2018. Learning to compare: relation network for few-shot learning. In *CVPR*.

[47] Canh T Dinh, Nguyen Tran, and Josh Nguyen. 2020. Personalized federated learning with moreau envelopes. *NeurIPS* (2020).

[48] Alysa Ziying Tan, Han Yu, Lizhen Cui, and Qiang Yang. 2022. Towards personalized federated learning. *IEEE TNNLS* (2022).

[49] Zhen Tan, Song Wang, Kaize Ding, Jundong Li, and Huan Liu. 2022. Transductive Linear Probing: A Novel Framework for Few-Shot Node Classification. *arXiv:2212.05606* (2022).

[50] Yonglong Tian, Yue Wang, Dilip Krishnan, Joshua B Tenenbaum, and Phillip Isola. 2020. Rethinking few-shot image classification: a good embedding is all you need? *ECCV* (2020).

[51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NeurIPS*.

[52] Oriol Vinyals, Charles Blundell, Timothy Lillicrap, Daan Wierstra, et al. 2016. Matching networks for one shot learning. In *NeurIPS*.

[53] Song Wang, Chen Chen, and Jundong Li. 2022. Graph Few-shot Learning with Task-specific Structures. In *NeurIPS*.

[54] Song Wang, Kaize Ding, Chuxu Zhang, Chen Chen, and Jundong Li. 2022. Task-adaptive few-shot node classification. In *SIGKDD*.

[55] Song Wang, Yushun Dong, Xiao Huang, Chen Chen, and Jundong Li. 2022. FAITH: Few-Shot Graph Classification with Hierarchical Task Graphs. In *IJCAI*.

[56] Song Wang, Xiao Huang, Chen Chen, Liang Wu, and Jundong Li. 2021. REFORM: Error-Aware Few-Shot Knowledge Graph Completion. In *CIKM*.

[57] Guandong Xu, Yanchun Zhang, and Lin Li. 2010. *Web mining and social networking: techniques and applications*. Vol. 6. Springer Science & Business Media.

[58] Qiang Yang, Yang Liu, Tianjian Chen, and Yongxin Tong. 2019. Federated machine learning: Concept and applications. *ACM TIST* (2019).

[59] Huaxiu Yao, Linjun Zhang, and Chelsea Finn. 2021. Meta-Learning with Fewer Tasks through Task Interpolation. In *ICLR*.

[60] Tao Yu, Eugene Bagdasaryan, and Vitaly Shmatikov. 2020. Salvaging federated learning by local adaptation. *arXiv:2002.04758* (2020).

[61] Qingyu Zhang and Richard S Segall. 2008. Web mining: a survey of current research, techniques, and software. *International Journal of Information Technology & Decision Making* (2008).

[62] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin, and Vikas Chandra. 2018. Federated learning with non-iid data. In *arXiv:1806.00582*.

[63] Zhuangdi Zhu, Junyuan Hong, and Jiayu Zhou. 2021. Data-free knowledge distillation for heterogeneous federated learning. In *ICLR*.

A NOTATIONS

In this section, we provide details for the used notations in this paper and their corresponding descriptions.

Table 2: Notations used in this paper.

Notations	Definitions or Descriptions
$\mathbb{S}$	the server
I	the number of clients
$\mathbb{C}^{(i)}$	the i-th client
C_b, C_n	the base class set and the novel class set
$\mathcal{D}_b^{(i)}, \mathcal{D}_n^{(i)}$	the local base and novel datasets in $\mathbb{C}^{(i)}$
$\mathcal{T}, \mathcal{S}, \mathcal{Q}$	a meta-task and its support set and query set
ψ, ϕ	the client-model and the server-model
$\mathbf{h}_\psi$	representations learned by client-model
$\mathbf{h}_\phi$	representations learned by server-model
$\mathbf{p}_\psi$	the output probabilities by client-model
$\mathbf{p}_\phi$	the output probabilities by server-model
N	the number of support classes in each meta-task
K	the number of labeled samples in each class
D	the number of support samples in each meta-task
Q	the number of query samples in each meta-task
α_{ft}	the learning rate for fine-tuning
α_ϕ, α_ψ	the meta-learning rates for ϕ and ψ
T	the number of training rounds
τ	the number of local training steps
λ_{MI}	the loss weight for the mutual information loss
λ_{KD}	the loss weight for the knowledge distillation loss

B ALGORITHM

We provide the detailed training process of our framework F^{2}L in Algorithm 1.

Algorithm 1 Detailed training process of our framework F^{2}L.

Input: A set of I federated clients; a local update objective $\mathcal{L}_\phi$ for server-model; a local update objective $\mathcal{L}_\psi$ for client-model; number of training rounds T; number of local training steps τ.

Output: A trained server-model ϕ and a unique client-model ψ_i for each client $\mathbb{C}^{(i)}$ in $\{\mathbb{C}^{(i)}\}_{i=1}^{I}$.

```
for t = 1, 2, ..., T do
    for each client C^(i) in {C^(i)}_{i=1}^I in parallel do
        for s = 1, 2, ..., τ do
            Sample a meta-task T_i^{t,s} = {S_i^{t,s}, Q_i^{t,s}};
            Fine-tune client-model ψ_i on S_i^{t,s} according to Eq. (3);
            Update server-model on S_i^{t,s} with Eq. (5) and Eq. (14);
            Update client-model on Q_i^{t,s} with Eq. (4) and Eq. (20);
        end for
    end for
    Each client returns the updated parameters of server-model
    to the server;
    The server sends back averaged parameters of server-model
    to each client;
end for
```

C REPRODUCIBILITY

C.1 Model Details

In this section, we introduce the specific choices for the encoders and classifiers in both server-model and client-model (i.e., q_ϕ, f_ϕ, q_ψ, and f_ψ).

C.1.1 Server-model Encoder q_ϕ. For the server-model encoder, we adopt different models for news article datasets and image datasets. In particular, for news article datasets 20 Newsgroup and Huffpost, we leverage a biLSTM [20] with 50 units as the server-model encoder. For the image datasets FC100 and miniImageNet, following [42, 50], we utilize a ResNet12 as the server-model encoder. Similar to [29], the Dropblock is used as a regularizer. The number of filters is set as (64, 160, 320, 640).

C.1.2 Client-model Encoder q_ψ. Considering that the client-model is required to process the entire support set in a meta-task for learning local meta-knowledge, we propose to further utilize a set-invariant function that takes a set of samples as input while capturing the correlations among these samples. In practice, we leverage the Transformer [51] as the client-model encoder q_ψ to process the entire support set:

$$\left(\mathbf{h}_\psi^1, \mathbf{h}_\psi^2, \ldots, \mathbf{h}_\psi^D\right) = \text{Transformer}\left(\mathbf{h}_\phi^1, \mathbf{h}_\phi^2, \ldots, \mathbf{h}_\phi^D\right), \tag{22}$$

where $\mathbf{h}_\phi^i$ (or $\mathbf{h}_\psi^i$) denotes the representation of the i-th sample in $\mathcal{S}$ learned by the server-model encoder q_ϕ (or client-model encoder q_ψ). With the Transformer, the representations learned by the client-model can effectively capture the correlations among samples in the entire support set $\mathcal{S}$ for learning meta-knowledge.

C.1.3 Server-model Classifier f_ϕ and Client-model Classifier f_ψ. The classifiers f_ϕ and f_ψ are both implemented as a fully-connected layer, where the output size is $|C_b|$ for f_ϕ and N for f_ψ, as described in Sec. 3.1.

C.2 Baseline Settings

In this section, we provide further details in the implementation of baselines in our experiments.

- *Local.* For this baseline, an individual model is trained for each client over the local data. Specifically, we use the same architecture of encoders in our framework to learn sample representations.
- *FL-MAML.* For this baseline, we leverage the MAML [15] strategy and set the meta-learning rate as 0.001 and the fine-tuning rate as 0.01. The encoders are the same as our framework.
- *FL-Proto.* For this baseline, we follow the setting in ProtoNet [44] with the same encoders in our framework. The learning rate is set as 0.001.
- *FedFSL* [14]. For this baseline, which combines MAML and an adversarial learning strategy [17, 43], we follow the settings in the public code and set the learning rate as 0.001. The adaptation step size is set as 0.01.

C.3 Parameter Settings

For our framework F^2L, we set the number of clients as 10. The number of training steps τ in each client is set as 10, and the number of training rounds T is set as 200. Moreover, the meta-learning rates α_ψ and α_ϕ are both set as 0.001 with a dropout rate of 0.1. The fine-tuning learning rate α_{ft} is set as 0.01. We leverage the Adam [26] optimization strategy with the weight decay rate set as 10^{-4}. During the meta-test, we randomly sample 100 meta-test tasks from novel classes C_n with a query set size $|Q|$ of 5. In order to preserve consistency for fair comparisons, we keep identical meta-test tasks for all baselines. The loss weights λ_{MI} and λ_{KD} are both set as 0.5. The default value of I is set as 10.